If the code fails to operate in your local tests, it will never work on a real robot.

Then there's the question of how you test your applications locally. You can test your ROS programmes without using a robot by using one of the numerous methods.

Levels of testing

Level 1. A library unit test (unittest, gtest) should test your code without using ROS (if you are having a hard time testing sans ROS, you probably need to refactor your library). Make sure your underlying functions are decoupled and thoroughly tested with both common and edge inputs. Make sure your methods' pre- and post- conditions have been thoroughly tested. Also, make sure your code's behaviour under these different scenarios is adequately described.

Level 2. ROS node unit test (rostest + unittest/gtest): Node unit tests start up your node and test its external API, which includes published topics, subscribed topics, and services.

Level 3. Integration/regression tests for ROS nodes (rostest + unittest/gtest): Integration tests start up numerous nodes and verify that they all operate together as intended.

Level 4. Functional testing: You should test the entire robot application at this level.

Check that the robot can navigate, grip, recognise humans, and so on... You'll need to come up with some tests to ensure that your code is still working on those features. The tests are usually carried out utilising simulations or bag files.

Creating tests for Levels 1, 2, and 3

In order to create tests for those levels, you will need to use the following libraries and packages:

- **Specifically for Level 1 tests:**
  - gtest: Also known as Google Test, it is used to build unit tests in C++. You can get the full documentation about using gtest with ROS at this link.
  - unittest: The framework provided for doing unit tests in Python. You can get a full documentation about using unittest with ROS at this link.
- **Additionally, for Level 2 and 3 tests:**
  - rostest: It is a package provided by ROS that allows the creation of